CIS 2400 – Phase 2 Game Design Document (GDD)

Title Section (Team & Project Information)

Project Information	A two player fighting game where users are able to interact with one another via keyboard inputs.
Project Team Name	Binary Bots
Team Members	Caleb Schell, Kevin Nguyen, and Haley Benge
Project Title (Game Name)	Runtime Rumble
Course & Phase	CIS 2400 – Phase 2 GDD
Date of Submission	March 20, 2026
Instructor / Evaluator	Dr. De Leoz

1. Project Overview

- **Game Title:** Runtime Rumble
- **Genre / Type:** Action Based, Fighting
- **Target Platform:** PC
- **Target Audience:** The Target audience is 13+ due to the use of violence in the game. The game would require minimal skill as it will be simple to learn. Players will be able to adjust the difficulty of the game based on their personal use of the fighting agents. Runtime Rumble is a player vs. player game.
- **Game Summary:** Players will use keyboard inputs to activate attack moves against each other. The goal of the game is to deplete your opponent's health bar before they are able to deplete yours using attack moves. Winning occurs when one player still has health status while the others has been fully depleted.

- **Unique Selling Point (USP):** Runtime Rumble is unique in its ability to distinguish what kind of attack move was used against a player and deplete their health based on this. For example, a "kick" may deplete a player's health faster than a "punch".

---

2. Game Concept & Design

- **Gameplay Loop:** Attack → Dodge → Increase/Decrease Health → Repeat
- **Player Actions:** Move (up, down, left, right), punch, kick, and duck.

Main Characters / Avatars & Movement

- **Appearance**:

Each player's avatar is a small humanoid figure with simple, blocky proportions and simplified limbs. When beginning the game players will be prompted to choose between 4 different avatars. The first being a dark-haired male wearing blue shorts and blue gloves, the second being a blond male wearing red shorts and red gloves, the third avatar will be a dark-haired female wearing a purple shirt, purple shorts, and purple gloves. The final avatar will be a blonde female wearing a pink shirt, pink shorts, and pink gloves.

- **Movement Mechanics:**

  Each character will be able to move left and right. Player one will use keys "a" to move left, "d" to move right, "w" to jump, and "ww" to duck. Player one will use "e" to punch and "ee" to kick. Player two will use keys "j" to move left, "l" to move right , "i" to jump and "ii" to duck. Player two will use "o" to punch and "oo" to kick. Their jump height is roughly half the size of the avatar, their duck height will be the same, and their left/right movements will be small in comparison. The first round of the game the players will move rather slow and then increase speed with each round.

- **Abilities & Interaction:** The players will be able to punch and kick their opponent. Each time a player makes contact with their opponent their health will increase and their opponents will decrease. A punch will decrease/increase the players health accordingly by 2%, while a kick will increase/decrease by 5%. Players will be able to hold multiple of their corresponding keys to make multiple movements at the same time, for example a player may move right and jump to avoid their opponents attack.
- **Win/Loss Conditions:** A player loses the game when their health bar is at 0% and their opponent is the winner. If the game is force ended before one players health reaches 0%, then the player with the highest percentage of health wins.
- **Unique Features / Mechanics:** Attack moves will affect health differently. Kicks will decrease the opponent's health more than punches.

- **Game Inspiration:** Street Fighter

---

3. Object-Oriented Design (OOP) – Java Focus

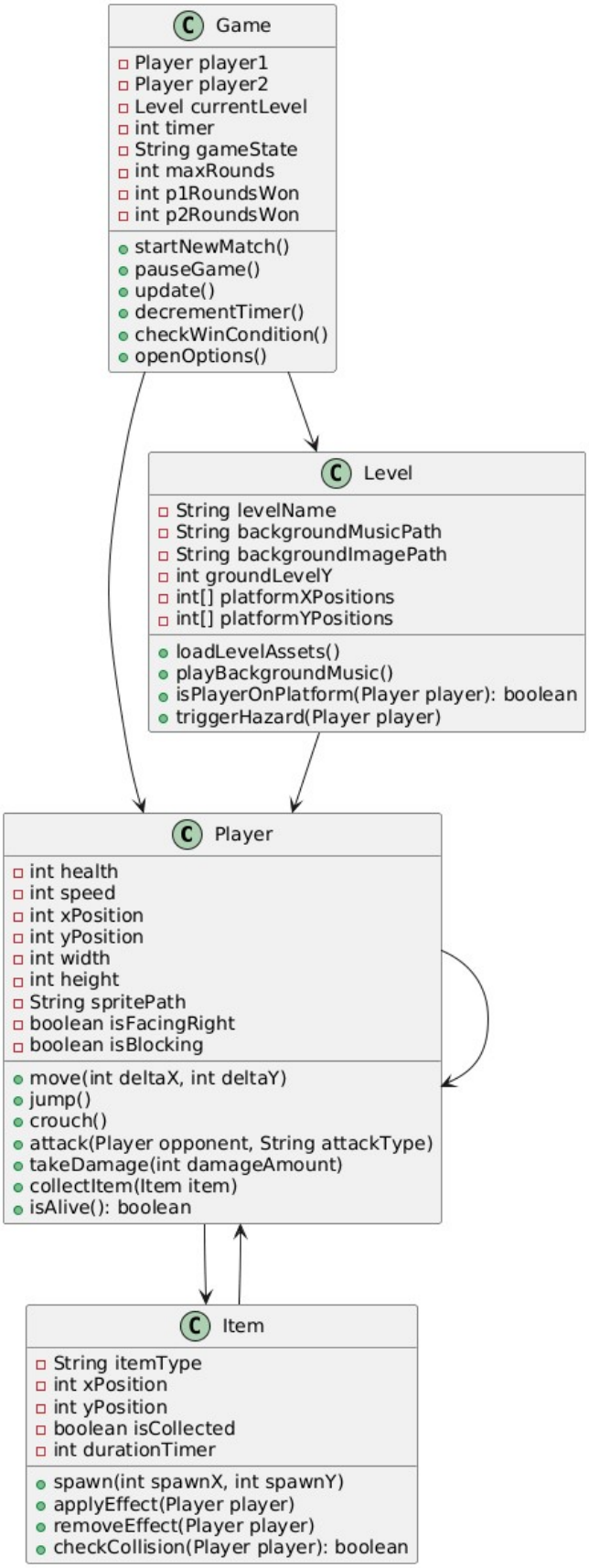
- **Class List & Purpose:**
  - Game – manages game flow, in-game timer, game status, options settings
  - Player – tracks health, position, speed, and appearance of both players
  - Item – speed and damage power-ups
  - Level – stores map layouts including platforms and hazards
- **Attributes & Methods:**

```
public class Player {
    private int health;
    private int speed;
    private int xPosition;
    private int yPosition;
    private int width;
    private int height;
    private String spritePath;
    private boolean isFacingRight;
    private boolean isBlocking;

    public void move(int deltaX, int deltaY) { ... }
    public void jump() { ... }
    public void crouch() { ... }
    public void attack(Player opponent, String attackType) { ... }
    public void takeDamage(int damageAmount) { ... }
    public void collectItem(Item item) { ... }
    public boolean isAlive() { return health > 0; }
```

```
}

public class Game {
    private Player player1;
    private Player player2;
    private Level currentLevel;
    private int timer;
    private String gameState;
    private int maxRounds;
    private int p1RoundsWon;
    private int p2RoundsWon;

    public void startNewMatch() { ... }
    public void pauseGame() { ... }
    public void update() { ... }
    public void decrementTimer() { ... }
    public void checkWinCondition() { ... }
    public void openOptions() { ... }
}

public class Item {
    private String itemType;
    private int xPosition;
    private int yPosition;
    private boolean isCollected;
    private int durationTimer;
```

```
    public void spawn(int spawnX, int spawnY) { ... }
    public void applyEffect(Player player) { ... }
    public void removeEffect(Player player) { ... } // Called when durationTimer runs out
    public boolean checkCollision(Player player) { ... }
}

public class Level {
    private String levelName;
    private String backgroundMusicPath;
    private String backgroundImagePath;
    private int groundLevelY;
    private int[] platformXPositions;
    private int[] platformYPositions;

    public void loadLevelAssets() { ... }
    public void playBackgroundMusic() { ... }
    public boolean isPlayerOnPlatform(Player player) { ... }
    public void triggerHazard(Player player) { ... }
}
```

- **Relationships & Interactions:** Both Players collect Item objects, Players interact with Level while moving, Player position is related to player input and Game physics, Player interacts with another Player object through damage exchange,
- **OOP Concepts Applied:** Encapsulation, Abstraction, and Composition
- **UML Diagram:**


Game
Player1
Player2
currentLevel
timer
gameState
maxRounds
p1RoundsWon
p2RoundsWon
startNewMatch()
pauseGame()
update()
decrementTimer()
checkWinCondition()
openOptions()
Level
levelName
backgroundMusicPath
backgroundImagePath
groundLevelY
platformXPositions
platformYPositions
loadLevelAssets()
playBackgroundMusic()
isPlayerOnPlatform(Player player): boolean
triggerHazard(Player player)
Player
health
speed
xPosition
yPosition
width
height
spritePath
isFacingRight
isBlocking
move(int deltaX, int deltaY)
jump()
crouch()
attack(Player opponent, String attackType)
takeDamage(int damageAmount)
collectItem(Item item)
isAlive(): boolean
Item
itemType
xPosition
yPosition
isCollected
durationTimer
spawn(int spawnX, int spawnY)
applyEffect(Player player)
removeEffect(Player player)
checkCollision(Player player): boolean

4. User Interface (UI) & Feedback System

- **Main Menu:**
  - Start menu with options for: Start Game, Settings, Exit, Player Appearance, and Level Selection. Use the mouse to select, executes option upon clicking.
- **HUD (Heads-Up Display):**
  - Player 1 health bar on the top left, Player 2 health bar pn the top right, countdown timer in the top middle, and a pause option in the bottom left corner.
- **Player Feedback:**
  - Player health changes from green to yellow to red based on the health bar remaining, sound plays when an item is picked up, sound plays when the level starts and ends along with "Start" and "Player (1 or 2) Wins" screen, Players make a sound when attacking. Left, right, jump, duck, punch, and kick, respectively
- **Controls:**
  - Player one will use keys "a", "d", "w", "ww", "e", "ee" to execute left, right, jump, duck, punch, and kick, respectively. Player two will use keys "j", "l", "i", "ii", "o", "oo" to execute left, right, jump, duck, punch, and kick, respectively. If an item is picked up, it is automatically applied, meaning that no action is required by the player for item use. The only GUI button shown during gameplay will be the "pause" button in the bottom left corner of the screen.

---

5. Pseudocode & Flowcharts

- **Pseudocode:**

```
while (game.isRunning) {
 Handle player input
   player1.move(deltaX1, deltaY1);
   player2.move(deltaX2, deltaY2);
Jump / crouch
   if (p1Jump) player1.jump();
   if (p2Jump) player2.jump();
```

```
    if (p1Crouch) player1.crouch();
    if (p2Crouch) player2.crouch();
  Attacks
    if (p1Attack) player1.attack(player2, attackType1);
    if (p2Attack) player2.attack(player1, attackType2);
Apply damage logic
    if (attackType1 == "punch") player2.takeDamage(2);
    if (attackType1 == "kick") player2.takeDamage(5);
     if (attackType2 == "punch") player1.takeDamage(2);
    if (attackType2 == "kick") player1.takeDamage(5);
Item interactions
    for (Item item : items) {
        if (item.checkCollision(player1)) item.applyEffect(player1);
        if (item.checkCollision(player2)) item.applyEffect(player2);
    }
Level interactions
    level.isPlayerOnPlatform(player1);
    level.isPlayerOnPlatform(player2);
    level.triggerHazard(player1);
    level.triggerHazard(player2);
Timer update
    game.decrementTimer();
Win condition
    game.checkWinCondition();
}
```

- **Flowchart:**

Flowchart

Start – Input – Move/Attack - Check Hit – Update Health - Health <= 0? - end/repeat

Repeat

---

6. Level / World Design

2D fighting arena that includes flat ground, optional platforms players start on both ends

- **Objects & Enemies Placement:**

Players: Spawn left/right sides

Items: Spawn at using coordinates spawn (x, y)

Difficulty & Progression:

- Each round increases player speed
- Lower time

- **Data Structures in Java:**

Player player1, player2;

ArrayList<Item> items;

---

7. Technical Design

- **IDE / Java Version:** IntelliJ IDEA, Java 17
- **Input / Output Handling**

Keyboard input (KeyListener) Graphics using Java Swing.JavaFX
Multiple inputs handleds for attacking and moving

Outputs include

Java Swing or JavaFX
Displays;
- Players movement
- Health bars
- Time and round information

- **Key Technical Challenges:**

Collision detection
- Must detect attack hits using the position of the player

Handing multiple key inputs
- Movement and attacking at the same time

Smooth health updates
- The health bar constantly updates

- **Code Organization / Modularity:**

Game – control loop
Player – movement + attacks

---

8. Potential Challenges & Next Steps

- **Biggest Challenges:**
- Implementing the attack mechanics so that kicks have a greater impact on health than punches, while making it harder for players to land kicks against their opponents.
- Balancing the game so that one player cannot "spam" attack their opponent and win the game immediately.
- Preventing the game from lagging or freezing due to the overload of updating both players health simultaneously at a quick pace
- **Next Steps / Planned Features:**
- Adding a background behind the players
- Integrate the HitsTaken into how the winner is defined
- Adding more attack option to round 2

---

9. References

- Gemini
- UML Visual Diagram Online Tool: https://www.planttext.com/
- ChatGPT
- Creating a Game with Java: A Comprehensive Guide — javaspring.net